

Q1)

1. Design:

```
1 module dff(clk, rst, d, q, en);
2   parameter USE_EN = 1;
3   input clk, rst, d, en;
4   output reg q;
5
6   always @(posedge clk) begin
7     if (rst)
8       q <= 0;
9     else
10      if(USE_EN) begin //Fixed Bug: Added Begain and End statments
11        if (en)
12          q <= d;
13      end
14      else begin //Fixed Bug: Added Begain and End statments
15        q <= d;
16      end
17  end
18
19 endmodule
20
```

2. Verification Plan:

Label	Design Requirment Description	Stimulus Generation	Funcational Coverage	Funcational Checking
D_FF_1	When the reset is asserted, the output q value must be low	Directed during the start of simulation		A checker in the testbench to make sure the output is correct
D_FF_2	Applying exhastive testing when en has value of 0	Directed during the simulation		A checker in the testbench to make sure the output is correct
D_FF_3	Applying exhastive testing when en has value of 1	Directed during the simulation		A checker in the testbench to make sure the output is correct

3. Testbench Code

Test Bench 1:

```
1 module dff_t1_tb();
2
3 parameter USE_EN = 0;
4 logic clk, rst, d, en;
5 logic q_dut;
6
7 int correct_count, error_count;
8
9 dff #(.USE_EN(USE_EN)) dut (clk, rst, d, q_dut, en);
10
11 initial begin
12     clk = 0;
13     forever
14         #1 clk = ~clk;
15 end
16
17 task reset_assert();
18     rst = 1;
19     check_result(0);
20     rst = 0;
21 endtask
22
23 task check_result(input logic q_expected);
24     @(negedge clk);
25     $display("Expected:%d | DUT:%d", q_expected, q_dut);
26     if(q_dut != q_expected) begin
27         error_count++;
28         $stop;
29     end
30     else begin
31         correct_count++;
32     end
33 endtask
34
35 initial begin
36     en = 0; d = 0;
37     error_count = 0;
38     correct_count = 0;
39
40     //D_FF_1
41     reset_assert();
42
43     //D_FF_2
44     en = 0 ;d = 1; check_result(1);
45     en = 0; d = 0; check_result(0);
46
47     en = 1 ;d = 1; check_result(1);
48     en = 1; d = 0; check_result(0);
49
50     //Additional Test Vectors for 100% Code Coverage
51     reset_assert();
52
53     en = 0; d = 0; check_result(0);
54
55     $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
56     $stop;
57 end
58
59 endmodule
```

Test Bench 2:

```
1 module dff_t2_tb();
2
3 parameter USE_EN = 1;
4 logic clk, rst, d, en;
5 logic q_dut;
6
7 int correct_count, error_count;
8
9 dff #(.USE_EN(USE_EN)) dut (clk, rst, d, q_dut, en);
10
11 initial begin
12     clk = 0;
13     forever
14         #1 clk = ~clk;
15 end
16
17 task reset_assert();
18     rst = 1;
19     check_result(0);
20     rst = 0;
21 endtask
22
23 task check_result(input logic q_expected);
24     @(negedge clk);
25     $display("Expected:%d | DUT:%d", q_expected, q_dut);
26     if(q_dut != q_expected) begin
27         error_count++;
28         $stop;
29     end
30     else begin
31         correct_count++;
32     end
33 endtask
34
35 initial begin
36     en = 0; d = 0;
37     error_count = 0;
38     correct_count = 0;
39
40     //D_FF_1
41     reset_assert();
42
43     //D_FF_2
44     en = 0; d = 0; check_result(0);
45     en = 0; d = 1; check_result(0);
46
47     //D_FF_2
48     en = 1; d = 0; check_result(0);
49     en = 1; d = 1; check_result(1);
50     //Additional Test Vectors for 100% Code Coverage
51     reset_assert();
52
53     en = 0; d = 0; check_result(0);
54
55     $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
56     $stop;
57 end
58
59 endmodule
```

4. Do File:

```
1  vlib work
2  vlog dff.v dff_t1_tb.sv +cover -covercells
3  vsim -voptargs=+acc work.dff_t1_tb -cover
4  add wave *
5  coverage save dff_t1_tb.ucdb -onexit -du work.dff
6  run -all
7  quit -sim
8  vlib work
9  vlog dff.v dff_t2_tb.sv +cover -covercells
10 vsim -voptargs=+acc work.dff_t2_tb -cover
11 add wave *
12 coverage save dff_t2_tb.ucdb -onexit -du work.dff
13 run -all
14 quit -sim
15 vcover merge dff_merged.ucdb dff_t1_tb.ucdb dff_t2_tb.ucdb -du dff
16 vcover report dff_merged.ucdb -details -annotate -all -output coverage_rpt.txt
```

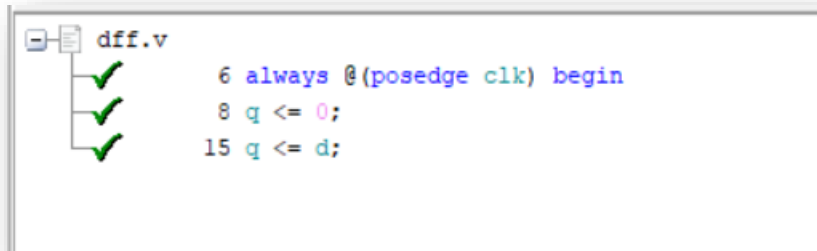
5. Code Coverage:

Test Bench 1:

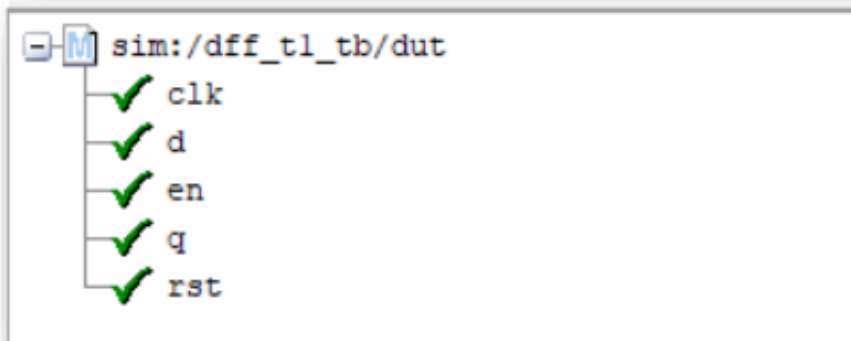
Branch Coverage:



Statement Coverage:

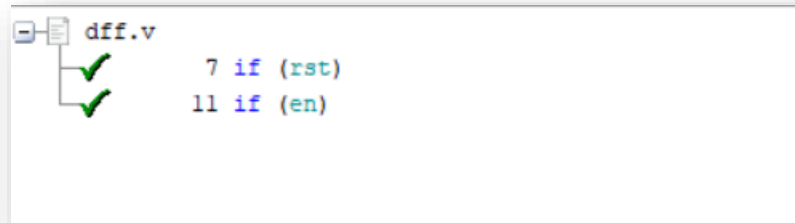


Toggle Coverage:

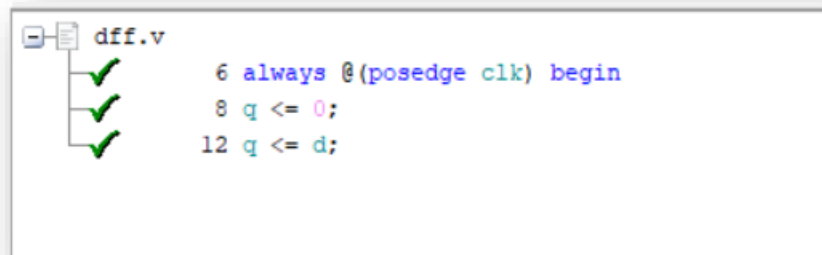


Test Bench 2:

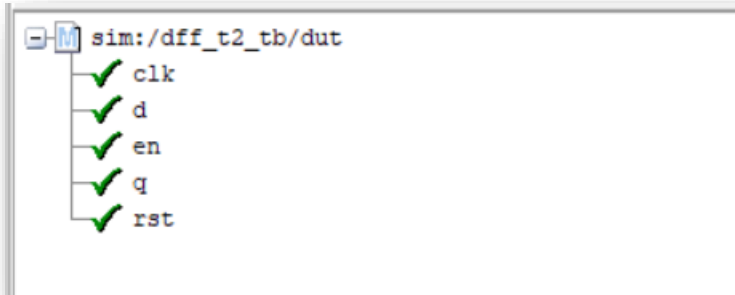
Branch Coverage:



Statement Coverage:

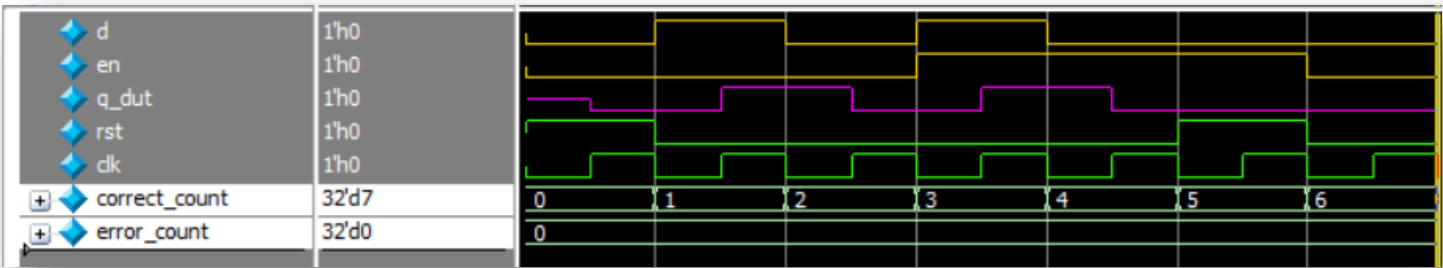


Toggle Coverage:

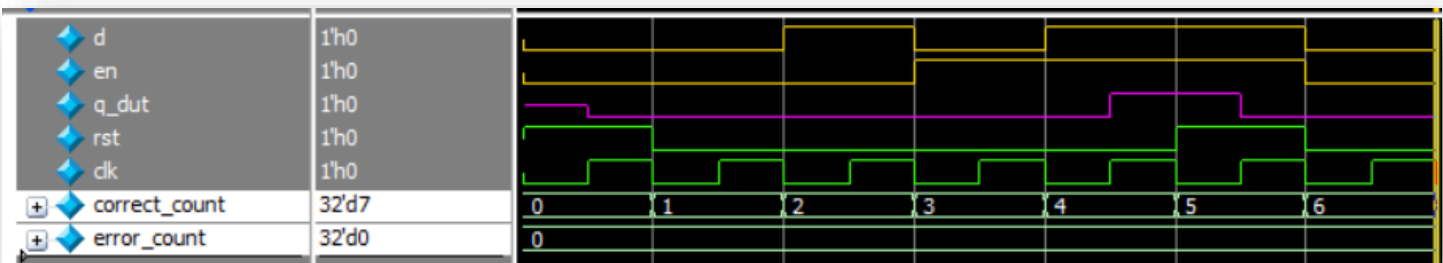


6. Questasim Waveform

Test Bench 1:



Test Bench 2:



7. Coverage Report:

Branch Coverage:

```
=====
=== Instance: /\work.dff
=== Design Unit: work.dff
=====
Branch Coverage:
  Enabled Coverage      Bins      Hits      Misses  Coverage
  -----
  Branches              4        4         0    100.00%

=====Branch Details=====
Branch Coverage for instance /\work.dff

  Line      Item      Count      Source
  ----      -
  File dff.v
  -----IF Branch-----
  7              14    Count coming in to IF
  7              4      if (rst)
  9              5      else
  11             2      if (en)
                               3    All False Count
Branch totals: 4 hits of 4 branches = 100.00%
```

Statement Coverage:

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	4	4	0	100.00%

=====Statement Details=====

Toggle & Total Coverage:

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	10	10	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\work.dff --

Node	1H->0L	0L->1H	"Coverage"
clk	2	2	100.00
d	2	2	100.00
en	2	2	100.00
q	2	2	100.00
rst	2	2	100.00

Total Node Count = 5
Toggled Node Count = 5
Untoggled Node Count = 0

Toggle Coverage = 100.00% (10 of 10 bins)

Total Coverage By Instance (filtered view): 100.00%